

Turning ADS-B state vectors into flight tracks

Version 1 — what the segmentation is worth, and what had to change
downstream before a better one could ship

EUROCONTROL Performance Review Unit

2026-09-03

Summary

The rule that built every `track_id` OPDI published up to v0.0.2 recovers about half the flights it should, the cause is one column in a grouping key rather than a mistuned threshold, and a rule that fixes it has now shipped.

Before OPDI can derive a flight list it has to decide which state vectors belong to the same flight. That decision — **segmentation** — is made once, and everything downstream inherits it: the flight list groups by `track_id`, events are extracted per track, measurements accumulate from a track's start. What follows is what the decision is worth, and what had to be repaired in the flight list before the better rule could be adopted without making the published product worse.

The old rule recovers about half the flights it should. On a three-day sample of 89,625 Network Manager flights it matches **50.15%** of them one-to-one, meaning one flight, one track, that track holding nothing else; **49.44%** of flights are split across more than one track. The failure is almost entirely that splitting, and almost never the opposite error of running two flights together. Count the turnaround as part of the flight, which is what a consumer of taxi phases needs, and the rate falls to **33.96%**.

The identity key is the problem. The old group key is (`icao24`, `callsign`), so a state vector decoded without a callsign forms a group — and therefore a track — of its own. Grouping on the airframe alone, and splitting only when the callsign *genuinely* changes, takes one-to-one matching to **90.80%** on 2025 and **91.75%** on 2024: a gain of **+40.65** points, and **+55.87** once the turnaround is scored as part of the flight. This is the rule that now ships, as `segmentation.method = "standard"`.

Its thresholds are the smaller question, and they were swept anyway. The shipped rule's three gap parameters were run over 235 combinations, every axis widened until the curve turned over inside the grid rather than at its edge. The settings that ship rank 22 and sit 0.18

points from the best cell anywhere on 2025 — and on 2024 the ordering of the one axis 2025 wanted to change reverses, so nothing was re-shipped.

The flight list could not consume it until one aggregate was fixed, and that is the most useful finding here. `flights.py` labelled each flight with the lexicographically smallest callsign in its track, and a blank sorts first — which is harmless only while every track carries exactly one callsign, a property the old group key supplied by accident and any fix to the fragmentation destroys. Against the pipeline as it then stood the better segmentation therefore made the published product *worse*, not by naming wrong aerodromes but by producing flights that could no longer be identified at all. That labelling is now fixed in production and the numbers on this page are the fixed ones.

With it fixed, the better segmentation costs **719** ADEP/ADES points on 2025 and **817** on 2024 — 0.54% and 0.63% of the total, where a *point* is the flight list’s own score for one correctly named aerodrome and a wrong one costs two — while recovering 41 points more flights. The residual is not accuracy, which is unchanged, and not departures, where the new rule is slightly ahead. It is arrival coverage, and the one failure mode the new rule makes worse is merging.

How to read this page

Sections are ordered as the work was done, not as the conclusion would suggest, because two of the results only make sense as answers to questions asked in order.

[What a track is](#) defines the object. [What has already happened to a sample](#) maps the pipeline steps that run before any segmentation sees a state vector; skip it if you know them. [How a segmentation is scored](#) defines every term the later sections use. [The sample](#) says what ground truth is. [The ladder](#) is one subsection per candidate rule. [The sweep](#) says what the shipped rule’s four parameters are worth. [The payoff](#) measures the downstream effect on ADEP/ADES and why it does not track segmentation quality. [What shipped](#) states what was adopted and what consumers have to know. [Limitations](#) is where the claims this page does *not* make are written down.

Every number here is an expression evaluated at render time over a committed CSV in `data/`, each regenerated by `benchmarks/regenerate_track_v1.py`, which the page runs before drawing anything and which stamps every output with the code that produced it in `data/_manifest.json`. Nothing is typed into the prose, which is what stops the text and the tables drifting apart. The exceptions are named where they appear: a few diagnostics measured while the arms were implemented, the gate-side block-time coverage, and two figures cited from other papers.

What a track is, and why the question was open

OPDI ingests OpenSky state vectors — position reports, one aircraft at a time, with no notion of a flight. A **track** is the pipeline’s answer to “which of these belong together”. The procedure that produces tracks is a **segmentation**, and a **track identifier** (`track_id`) is what it stamps on every sample it decides belongs to the same flight.

The rule this study set out to evaluate — called **legacy** throughout, and the rule behind every dataset OPDI published up to v0.0.2 — is short. Group the samples by (`icao24`, `callsign`), order them by time, and start a new track when the gap to the previous sample exceeds 30 minutes, or exceeds 15 minutes while the aircraft is below 5,000 ft. The resulting identifier carries a `_year_``_month` suffix.

Three things can go wrong with such a rule, and they are not symmetric:

- **Fragmentation** — one real flight becomes several tracks. Everything downstream sees several short flights where there was one.
- **Merging** — several real flights become one track. The flight list emits one flight and the others simply do not exist in the output.
- **Boundary error** — the track is one flight, but starts or ends in the wrong place, so first- and last-seen events and cumulative measurements are wrong.

Merging is the most damaging of the three, because a merged flight is unrecoverable downstream while a fragmented one is at least present in pieces. The metrics below therefore keep merging and fragmentation separate rather than blending them into one score. The rule had never been benchmarked before this study, which is why the study exists.

Segmentation is a versioned choice, not a frozen one: changing the rule changes `track_id` for everything produced from that release forward, in value and in shape where the month suffix goes. That decision has now been taken. `segmentation.method` defaults to `"standard"`, the rule this study recommends, and `"legacy"` remains selectable and reproduces the identifiers of every dataset published up to OPDI v0.0.2.

This paper measures algorithms, not the pipeline. Every rule below is expressed in a generic segmentation engine and scored in a harness, so that eight candidates could be compared without eight production pipeline runs — at this data volume, the difference between a study and a quarter. What is measured is therefore a re-implementation of each rule rather than the pipeline’s own output. Version 2 of this study runs the pipeline itself, end to end, on the rule this one recommends.

What has already happened to a sample

Side information: if you know the pipeline steps, skip to [the metrics](#).

Segmentation is step 02 of a chain, and the samples reaching it are not raw OpenSky rows. Two earlier decisions in particular shape everything measured on this page: the ingestion filter decides which samples exist at all, and the **thinning** rule — which sample survives out of each short interval — decides how densely a flight is described.

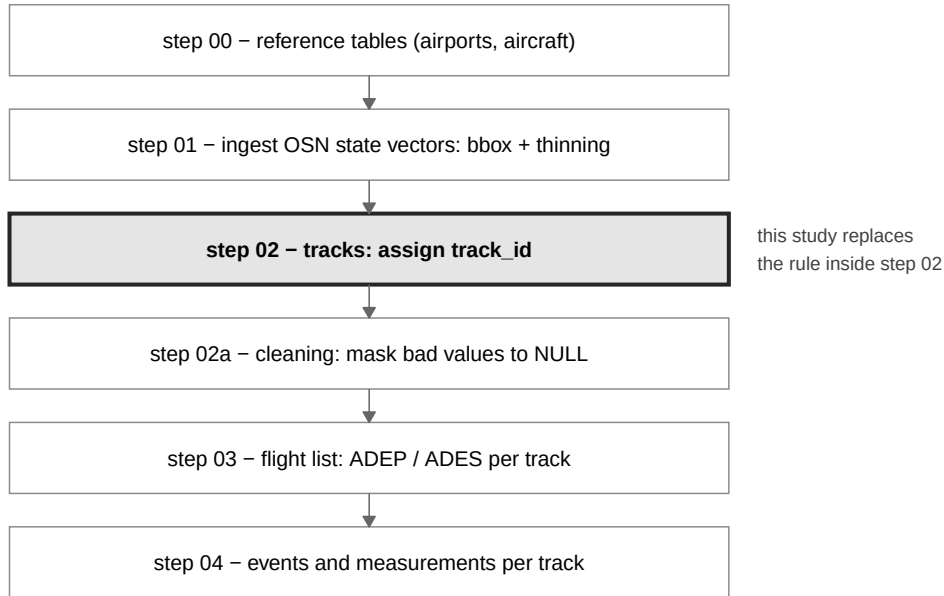


Figure 1: The pipeline steps that run before, and after, the decision this page studies. Step 02 is the one under test; everything above it has already happened to a sample by then, and everything below it inherits the result.

Step	What it does	Writes
00	Builds reference tables: H3 airport detection zones, HexAero airport ground layouts, OurAirports, the OpenSky aircraft database. Read later; never rewritten by a pipeline run.	reference tables
01	Downloads OpenSky state vectors, restricts them to the OPDI bounding box, and thins them to one sample per aircraft per interval.	<code>osn_statevectors</code>
02	Groups samples into tracks and stamps <code>track_id</code> . This is the step under study. Also attaches the H3 index each later join uses.	<code>osn_tracks</code>
02a	Masks values that cannot be right — duplicates, out-of-range positions, stale broadcasts, derivative spikes, isolated points — to NULL, keeping the row. Writes a second table rather than editing the first.	<code>osn_tracks_clean</code>

Step	What it does	Writes
03	One flight per <code>track_id</code> : departure and destination aerodrome, and the flight's callsign label.	<code>opdi_flight_list</code>
04	Milestones per track — take-off, top-of-climb, level segments, landing — and the measurements attached to them.	events, mea- sure- ments

Every count on this page is conditioned on three parameters of step 01.

Parameter	Value	Why it matters here
bounding box (min lon, min lat, max lon, max lat)	(-25.86653, 26.74617, 49.65699, 70.25976)	A flight is observed only inside this box. A track that begins at its edge may have entered from outside, which is why an out-of-area label exists at all.
thinning interval	5 s	Consecutive samples of one aircraft are at least this far apart, so every gap threshold in this study is measured against a 5 s grid rather than a 1 Hz one.
thinning rule	<code>bucket</code>	One row kept per aircraft per interval bin — the last, so a track's final observation survives exactly. The earlier <code>modulo</code> rule kept the row whose epoch second divides the interval, and built every dataset published before it.

The diagram's ordering is production's, and this study reverses it. In production step 02 builds tracks and step 02a cleans them; here every candidate rule is applied to the *cleaned* table, so that all of them see the same input and a difference between two arms cannot be an artefact of different cleaning.

That has one consequence. Cleaning masks bad values to NULL rather than deleting rows, and `NULL < 5000` is NULL rather than true, so legacy's low-altitude split does not fire on a sample whose altitude the cleaner rejected, where once it did. Every arm is subject to that equally, so no comparison here is affected — but the identifiers in published data cannot be recomputed from the published clean table, which [Limitations](#) picks up.

How a segmentation is scored

Nothing after this section introduces a metric.

Comparing a segmentation against ground truth is a **clustering comparison**: both the algorithm and the truth partition the same state vectors — the algorithm into tracks, the truth into flights — and the question is how far the two partitions agree.

Homogeneity is merging; completeness is fragmentation

Two standard clustering measures map onto the two failure modes exactly, which is why they are used rather than a score invented for the occasion:

- **Homogeneity** asks whether each track contains samples from only one flight. It falls when a track holds more than one flight — so it is the *merging* measure.
- **Completeness** asks whether each flight's samples all landed in one track. It falls when a flight is spread over several tracks — so it is the *fragmentation* measure.

Both are entropy-based and both run from 0 to 1, with 1 meaning perfect agreement with ground truth on that axis alone. Both are also entropy-weighted over every sample in the period, and the overwhelming majority of samples sit in tracks that every candidate gets right: across the eight rules they span 0.9935–0.9999 and 0.9936–0.9984 respectively, where the one-to-one matching rate defined next spans 37.6 to 91.8 points. They name the two failure modes; they do not rank the candidates. That is what the next statistic is for.

The headline: one-to-one matching

The headline statistic on this page is **clean_match_pct**, and it answers the question a data consumer actually has: *what share of real flights came out as exactly one track, holding nothing but that flight?*

Every ground-truth flight is placed in exactly one of three mutually exclusive classes, which sum to 100%:

- **clean** — the flight's samples are in one track, and that track carries no other flight;
- **fragmented** — the flight's samples span several tracks, none of which carries another flight;
- **merged** — the flight's dominant track also carries at least one other flight.

A flight that is both merged and fragmented counts as **merged**, because merging is the worse failure. A single stray sample in a second track costs a flight its clean match,¹ so **clean_match_pct** is a lower bound and is reported as one.

Matching a track to a flight: interval containment

Every statistic above needs a rule saying which ground-truth flight a given state vector belongs to. That rule is **interval containment**, and it is set out fully because it is where a comparison of this kind is most easily rigged without anyone noticing.

¹The tolerant alternative — a dominant track covering some fraction of the flight — was rejected because that fraction is a free parameter chosen after seeing the results.

A ground-truth flight is an airframe and a span of time. The airframe is `icao24` — the same 24-bit address ADS-B broadcasts, which the Network Manager flight table carries as `AIRCRAFT_ADDRESS`. Two spans are defined, and every arm is scored against both. The first is the flight’s airborne interval, from take-off to landing:

- where APDF covers the aerodrome, from APDF’s measured movement times, the real ATOT and ALDT;
- elsewhere, inferred from Network Manager as `AOBT_3 + TAXI_TIME_3` for take-off and `ARVT_3` for landing. Both inferences were validated against real APDF movements before being relied on (see [The sample](#)).

Each row carries which of the two it came from, so a statistic that needs measured times rather than inferred ones — boundary error does — can restrict itself to the APDF-backed subset.

The join is on `icao24` and time overlap, and never on `callsign`. This differs from the ADEP/ADES flight-list studies, which key on `callsign`, and the difference is forced rather than stylistic: one of the candidate rules below removes `callsign` from track identity entirely, and another splits on it. Joining on `callsign` would therefore score each candidate against a key it deliberately does or does not have, which is prejudging the very thing under comparison.

Containment, concretely. A state vector is attributed to the ground-truth flight of the same `icao24` whose interval holds its timestamp. A track and a flight are then *matched* when at least one of the track’s samples is attributed to that flight, and the contingency of tracks against flights is what the three classes above are counted from. Where two intervals of the same airframe touch, the boundary sample goes to the earlier one, so back-to-back legs cannot double-count it and inflate the merge rate.

A worked example, one airframe with two rotations in an afternoon:

Ground-truth flight	Its interval	A sample	Attributed to
leg 1	12:00–13:30	12:05	leg 1
leg 1	12:00–13:30	13:25	leg 1
gap	—	13:50 (on stand)	no flight — outside both intervals
leg 2	14:10–15:40	14:15	leg 2
leg 2	14:10–15:40	15:35	leg 2

If one track holds the 12:05 and 15:35 samples, both flights are **merged** into it. If leg 1’s two samples land in different tracks, leg 1 is **fragmented**. The samples on stand at 13:50 belong to no flight and are scored against nothing. That is the airborne metric’s blind spot, and the second interval, [below](#), is what measures it.

What the restriction costs. Ground truth is further restricted to flights whose *whole* airborne interval lies inside the sampled window: a flight observed only for its last twenty minutes would score as truncated however well the segmentation performed, and its boundary

error would be a fact about where the window fell. That argument justifies the restriction without pricing it, so the price was measured.

period	any airborne time in window	wholly inside	clipped at start	clipped at end	kept %	median observed fraction of an excluded flight
2025	97,088	93,035	1,954	2,099	95.83	0.504
2024	94,686	90,879	1,853	1,954	95.98	0.502

The restriction keeps 95.83% of the 2025 flights with any airborne time in the window and 95.98% of the 2024 ones, and the exclusion is near-symmetric between the two window edges, which is what a window artefact should look like rather than a rule that systematically drops long-haul arrivals.

Nor is it discarding nearly-complete flights on a technicality. The median excluded flight had 50% of its airborne time inside the window on 2025 and 50% on 2024 — about half observed, and unscoreable.

One further gap explains a difference between two counts on this page. Of the 93,035 flights wholly inside the 2025 window, 89,625 are scored — the remaining 3,410 produced no ADS-B sample at all in the feed and so appear in no contingency. They are absent from the denominator for the same reason, and no candidate rule is credited or penalised for them.

The second interval: gate to gate

Containment as defined above matches on the airborne interval, so a sample taken on stand or on a taxiway is attributed to no flight and leaves every metric where it found it. That defensible line hides the failure a data consumer meets first. An OPDI track is supposed to carry the turnaround: first- and last-seen events fire on it, taxi phases are extracted from it, and every cumulative measurement starts at its first sample. A rule that shatters a flight into three tracks between the gate and the runway threshold scores a clean airborne match and hands over a broken product. So each arm is scored twice over the same partition: once against the airborne interval, once against the flight’s **gate-to-gate interval**, off-block to in-block.

Where the block times come from. aobt prefers APDF’s measured off-block time and falls back to Network Manager’s AOBT_3, so it exists for substantially every flight. aibt has no fallback and cannot have one — ARVT_3 is a landing time, not a gate arrival — so the arrival side is APDF-only, and 61.2% of scored flights carry a measured in-block time against 100.0% on the departure side. Where a block time is missing the interval is extended past the airborne endpoint by the median measured taxi on that side, over the flights of that period that did measure one — 720 s before take-off and 310 s after landing on the 2025 sample. Taking the modelled half’s duration from the measured half is the honest version of a buffer; ten minutes because it sounds like a taxi is not.

The gate interval is clamped to contain the airborne one. APDF carries rows whose block time falls the wrong side of its own movement time. Unclamped, such a row gives a gate interval *narrower* than the airborne one, and the wider metric would drop samples the narrower one kept. Clamped, the two differ only by the samples the gate interval adds, which is what makes the difference between the two rates readable as one quantity.

One qualification is exposed rather than filtered away. The sample window bounds the airborne interval and cannot bound the gate interval, which is attached after that filter, so a flight can be wholly in-window airborne and still push back before the window opens. About a third of a percent are, and ground truth carries the condition per flight as `gate_in_window`. Their taxi samples are clipped by the day filter, in the flattering direction, since fewer samples is fewer chances to fragment. Filtering on the column would change which flights the gate metric covers without saying so, so it is carried and not applied.

The two metrics score 89,626 and 89,625 flights on 2025, one flight apart, so nothing below is the wider interval scoring a different population.

Boundary error, reported signed

Boundary error is the distance in time from a track's first and last sample to the flight's actual take-off and landing. It is reported **signed**, with the convention that

off = track start — take-off, land = track end — landing. A negative value means the track began (or ended) *before* the aircraft did.

That is not cosmetic. An absolute error conflates two opposite conditions. A track that starts *before* take-off has picked up the taxi-out, and that overhang is wanted rather than tolerated: those samples are the flight's own, [the gate interval](#) scores them, and the taxi phases OPDI publishes are extracted from them. A track that starts *after* take-off has missed part of its own flight. `legacy`'s absolute median is about a hundred seconds, a small tidy number. Signed, its median is **+6 s** and its tenth percentile **-806 s**: the typical track begins just after wheels-off, while a minority carry a full quarter-hour of taxi-out. An absolute median over a two-sided distribution is close to meaningless.

The landing side is one-sided, every percentile positive, median **+359 s**. That asymmetry is a coverage fact rather than an algorithm defect: on-ground ADS-B before departure reaches a minority of flights, while an arriving aircraft is still powered and broadcasting to the stand.

The percentiles are computed over the flights whose ground-truth times are APDF-measured rather than NM-inferred — 35,398 of the 89,625 scored flights on 2025 — because a boundary error against an inferred time would partly be measuring the inference.

Table 1: Signed boundary error in seconds, 2025. Negative means the track began or ended before the aircraft did.

arm	dep p10	dep p50	dep p90	arr p10	arr p50	arr p90
airframe_only	-891	-1	94	115	362	799
recommended	-838	-1	109	67	359	768
airport_anchored	-831	4	94	116	362	800
traffic_style	-917	5	94	120	371	828
legacy	-806	6	109	105	359	742
no_month_suffix	-806	6	109	105	359	742
ground_anchored	-22	19	94	115	360	780
vertical_profile	19	53	114	69	360	797

The shape is what the percentiles cannot show.

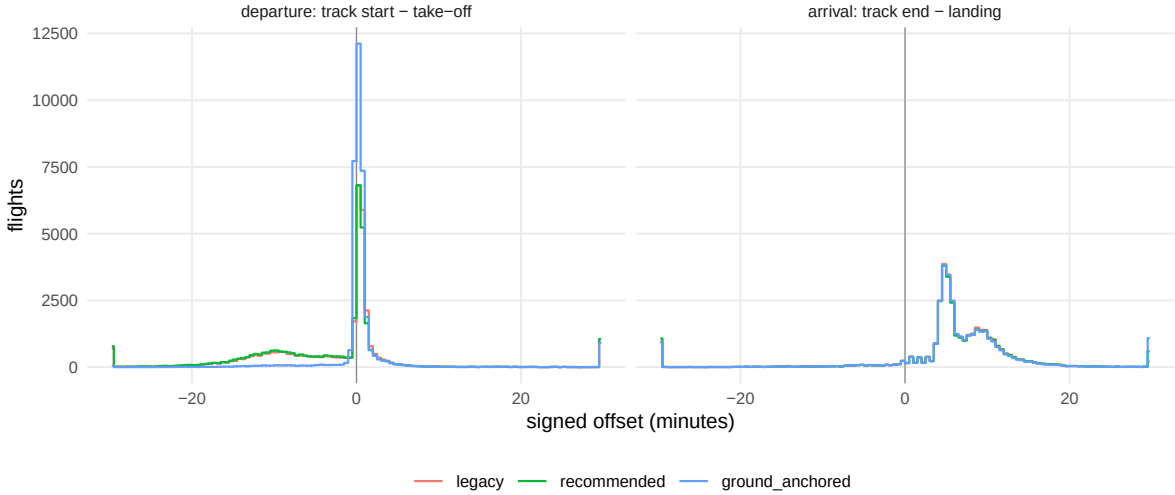


Figure 2: Signed boundary error on the 2025 sample, over the flights whose ground-truth times are APDF-measured. **Negative means the track began, or ended, before the aircraft did.** Departure: `legacy` and `recommended` both show a two-sided distribution — a spike just after wheels-off plus a long left tail of tracks that started on the ramp — while `ground_anchored` concentrates hard at zero, which is the arm cutting at the ground event rather than running back into the ramp. Arrival: all three are one-sided and nearly identical, because tracks essentially always run on through taxi-in. The outermost bin on each side is an overflow bin and carries everything beyond half an hour.

The sample

Two three-day samples, 89,625 and 87,071 scored ground-truth flights: 5–7 June 2025 and 5–7 June 2024. Two periods rather than one, because a single sample cannot distinguish a real effect from one that sample happened to favour. Every conclusion on this page has to hold on both.

Ground truth is the Network Manager flight table, which covers all of ECAC, cross-checked against APDF where APDF covers the aerodrome. Take-off is inferred as `A0BT_3 + TAXI_TIME_3` — off-block plus taxi-out — and landing is taken from `ARVT_3` directly, which is already a landing time rather than a gate arrival. Both inferences were validated against real APDF movement times before being relied on, at median error 0 s with an interquartile range of 17 s and 25 s respectively; that is what licenses using NM-inferred intervals for matching. Boundary error is nevertheless reported only over the APDF-measured subset, which is conservatism rather than necessity.

The ladder: eight candidate rules

A **ladder** here is a set of candidate segmentation rules run against the same ground truth, on the same samples, one change at a time. Each candidate is called an **arm**, borrowing the term from a controlled trial, and each arm is exactly two things: a **grouping** — which samples are even eligible to share a track — and a **break predicate** — a test evaluated at each sample in time order, which when true starts a new track.

Every arm reads one shared cleaned track table, and only `track_id` varies between them, so a difference between two arms cannot be an artefact of a different track build. Arms are numbered A0 to A8. A1 was never an arm: it was a parameter sweep, and [it now sweeps the rule that ships](#) rather than the one that does not.

Every domain-knowledge arm (A5, A6, A7) keeps a legacy gap rule as a **floor**: it can add splits to legacy’s, and in one deliberate exception replace one of them, but it cannot split less than a long silence demands. Two reasons, both of which matter for reading the numbers. The ladder measures one change at a time, so an arm that also silently removed a legacy split would report a delta mixing two changes; and an arm whose inputs are missing must not degrade into assigning an airframe’s whole month to one track.

arm	clean %	gate clean %	gate — air	fragmented %	merged %	tracks	clean (2024)
recommended	90.80	89.83	-0.97	8.45	0.75	98,876	91.75
airframe_only	88.56	88.51	-0.05	6.90	4.53	94,130	89.79
airport_anchored	86.15	83.52	-2.63	9.55	4.30	96,717	89.03
ground_anchored	80.11	61.80	-18.30	18.72	1.17	106,621	78.64
legacy	50.15	33.96	-16.19	49.44	0.41	146,797	52.79

arm	clean %	gate clean %	gate — air	fragmented %	merged %	tracks	clean (2024)
no_month_suffix	50.15	33.96	-16.19	49.44	0.41	146,797	52.79
traffic_style	46.37	31.89	-14.48	44.14	9.50	111,027	48.18
vertical_profile	37.62	36.72	-0.89	59.91	2.47	145,774	38.93

The ordering is identical on both periods, and no arm’s clean-match rate moves by more than 2.9 points between them, so what follows is a property of the algorithms rather than of one sample.

The gate columns are the [second interval](#), and they widen the result rather than repeat it. **recommended** beats **legacy** by **+40.65** points airborne and **+55.87** gate to gate on 2025, and by **+38.96** and **+56.00** on 2024. Scoring only the airborne leg understated what the shipped change is worth by about 15 points.

What the wider interval detects is a property of the group key, not of the break predicate:

Table 2: What each arm loses when the turnaround is scored as part of the flight.

arm	group key	gate — air (2025)	gate — air (2024)
ground_anchored	icao24	-18.30	-16.74
legacy	icao24 + callsign	-16.19	-17.99
no_month_suffix	icao24 + callsign	-16.19	-17.99
traffic_style	icao24 + callsign	-14.48	-15.76
airport_anchored	icao24	-2.63	-0.48
recommended	icao24	-0.97	-0.95
vertical_profile	icao24	-0.89	-0.74
airframe_only	icao24	-0.05	-0.04

Every arm carrying callsign in its group key loses between fourteen and eighteen points. A callsign goes blank, or flips to the next rotation’s value, while the aircraft is parked — so a key containing it splits the taxi phases into separate tracks, in exactly the place the airborne metric cannot see. **ground_anchored** loses as much for a different reason: it cuts through the turnaround by construction. The arms that group on the airframe alone lose under a point, because nothing they do fires at the stand. **airport_anchored** is the one arm whose drop moves between periods, -2.63 against -0.48: it groups on the airframe, but its stationary-at-an-aerodrome break can fire during the dwell the gate interval scores, and how often it does depends on which aerodromes the sample contains.

Each arm below has the same four parts: what it changes relative to **legacy**, what it is trying to fix, the rule, and what the measurement said.

A0 legacy — the baseline

What it changes. Nothing. This is the rule that built every published `track_id` up to v0.0.2, and everything else is measured against it.

The rule. Group the samples by (`icao24`, `callsign`) — in production, by a SHA-2 hash of the two concatenated. Order each group by time. Break when the gap to the previous sample exceeds 30 minutes, or exceeds 15 minutes while barometric altitude is below 5,000 ft. The identifier carries a `_year_``_month` suffix.

What the measurement said. 50.15% clean, 49.44% fragmented, 0.41% merged on 2025; 52.79% clean on 2024. The failure is overwhelmingly fragmentation, and almost never merging.

The cause is not the obvious one. It is *not* that callsigns change in mid-flight; it is that **a sample with no callsign forms a group, and therefore a track, of its own**. Measured while the arm was implemented, 42.4% of `legacy`'s tracks carry no callsign at all — they are made of samples the receiver decoded without one. The group key turns a hole in the data into a separate flight, and that single mechanism is what the rest of this ladder chases.

A2 no_month_suffix — untestable on this sample

What it changes. Removes the `_year_``_month` suffix from the identifier. Grouping and break predicate are `legacy`'s, untouched.

What it is trying to fix. The suffix splits any flight airborne at midnight on the first of a month, by construction and regardless of the data. That is a correctness defect rather than a threshold, which is why no amount of parameter sweeping can reach it.

What the measurement said. Identical to A0 in every digit, on both periods, which is correct behaviour rather than a bug: both samples sit inside a single month, so the suffix is constant. **This study cannot evaluate the arm at all**, and reports it as untested rather than as a null result. Testing it needs a sample spanning a month boundary, about two days of cluster time.

A3 traffic_style — the reference library's rule, outside its own preprocessing

What it changes. Replaces `legacy`'s two hard thresholds with the semantics of `Flight.split` in the *traffic* library, the widely used open-source reference for this kind of work.

What it is trying to fix. Nothing specific to OPDI. The question this arm answers is whether the established reference implementation, applied to this feed, would simply have done better.

The rule. Group on (icao24, callsign). Break when the gap exceeds a single threshold — *traffic*’s documented default is ten minutes — *unless* a predicate says the candidate gap is not really a break. The predicate used here is the aviation-sensible inverse of *traffic*’s own documented example: do not split when the samples on both sides of the gap are above the low-altitude ceiling, because a reception hole in the cruise is a coverage gap, not a landing.

What the measurement said. 46.37% clean, below `legacy`, and the worst merging of any arm at 9.50% — about 23 times `legacy`’s. Two things are behind that, and only the second is the reason it lost.

The predicate reads a column OPDI has taken away

`Flight.split` fills nothing: it cuts on gaps and hands back the pieces. It is one method of a class whose neighbours do fill, though — `Flight.filter` defaults to `bfill().ffill()` and `Flight.resample` to interpolation — and a *traffic* pipeline conventionally runs one of them first. The predicate is written for a frame with no holes in it.

OPDI hands it a frame that is almost all hole in the one place the predicate looks. The test asks whether the samples either side of a candidate gap are above the low-altitude ceiling, and `NULL > 10000` evaluates to `NULL` rather than to `false`, so a boundary with a masked altitude on either side is one the arm cannot decide.

period	candidate gaps	undecidable, cleaned %	undecidable, raw %	turnarounds with no gap at all
2025	129,893	99.13	35.18	6,460
2024	121,951	99.58	35.64	4,946

On the cleaned table the arm reads, 99.13% of candidate gaps on 2025 and 99.58% on 2024 are undecidable. On the raw table the same count is 35.18% and 35.64% — a third rather than everything. **The missingness is not in the feed; OPDI’s cleaning makes it**, and one rule in particular. `mask_stale_broadcasts` masks an altitude that exactly repeats the previous sample’s, because ADS-B sends position and velocity in separate message types and an unchanged consecutive value is a repeat rather than a measurement. A coverage fade loses fresh altitude before it loses messages altogether, so the last sample before a silence is precisely the sample most likely to be a repeat: on 2025, 30.44% of pre-gap samples are already `NULL` in the raw feed and 99.1% of the rest repeat their predecessor, against 9.39% and 55.69% away from a boundary. The masking is correct and the predicate is correct. They cannot be used together.

That is not why the arm lost

A3’s failure is merging, and merging is under-splitting. Filling the altitude column could only let the predicate *decline* to split; nothing it does can manufacture a split that was not offered. And the splits A3 needs were never offered: 6,460 turnarounds on 2025 and 4,946 on 2024 produce no candidate gap at all, because the aircraft broadcast straight through the stand. No fill reaches them. Legacy’s second rule — the shorter threshold below 5,000 ft — is what catches them, and a single gap threshold has no equivalent.

So the conclusion is narrower than “the reference approach does not transfer”, and neither half of it is a defect in the reference. The rule is used outside the preprocessing it assumes, and even inside that preprocessing one threshold cannot see a turnaround that never went quiet.

A4 airframe_only — delete callsign from the group key

What it changes. One column. Group on icao24 alone; callsign becomes an attribute of a sample rather than part of the track’s identity. The break predicate is **legacy’s**, untouched.

What it is trying to fix. A0’s diagnosis directly: if a blank callsign is what starts a spurious track, removing callsign from the key stops it. Identity belongs to the airframe; the callsign is what the airframe happened to be broadcasting at the time.

What the measurement said. 88.56% clean, with fragmentation down from 49.44% to 6.90%. This is the single largest movement anywhere in the study, and it comes from deleting one column.

But it is not free, and the cost is exactly the failure legacy did not have: merging rises from 0.41% to 4.53%. Without callsign in the key, two consecutive flights of the same airframe with no sufficient gap between them become one track. Measured during implementation, 3.3% of this arm’s tracks contain more than one genuine callsign.

A5 ground_anchored — split at a turnaround, using the on-ground flag

What it changes. Groups on icao24, like A4, and *adds* a break condition to legacy’s gap rules rather than replacing them.

What it is trying to fix. Legacy’s “below 5,000 ft” test is a proxy for “the aircraft is on the ground somewhere”, used because barometric altitude was all the rule had. The OSN schema carries an **on_ground** flag that step 02 reads nothing of, and this arm uses it.

The rule. A break is declared at the *first airborne sample after* a run of consecutive on-ground samples lasting at least the ground-dwell threshold. The dwell requirement is what separates a turnaround from a touch-and-go. Placing the break at the first airborne sample, rather than at the start of the ground run, means the departing leg begins at the rotation

instead of inheriting the previous arrival's taxi-in. Legacy's two gap rules are kept as a floor, because an aircraft that stops broadcasting entirely still has to be split somehow.

What the measurement said. 80.11% clean — well above `legacy`, well below A4. Its distinguishing result is not the headline but the boundary error: its departure tenth percentile is -22 s against legacy's -806 s, and the figure in [the metrics section](#) shows the whole left tail collapsing. The arm cuts at the ground event rather than running back into the ramp.

Scored [gate to gate](#) that same behaviour is its worst result: 61.80%, a drop of -18.30 points, the largest of any arm on that sample. Cutting at the ground event means cutting through the turnaround, so one flight's taxi-in and taxi-out land in different tracks. The arm locates rotation precisely and delivers a flight poorly.

A6 airport_anchored — split only where an aerodrome actually is

What it changes. Groups on `icao24`, and replaces legacy's low-altitude gap rule with real airport geometry. It is the one arm that takes only *half* of legacy as its floor.

What it is trying to fix. Two errors that no altitude threshold can reach. A split legacy *misses*: an aircraft parked at a 6,000 ft aerodrome never satisfies “below 5,000 ft”, so the turnaround is invisible and two flights merge. And a split legacy *invents*: a long slow low transit in the middle of nowhere satisfies the low-altitude rule and is split, when nothing happened at all.

The rule. A sample is *stationary at an aerodrome* when it is inside a known airport detection zone, below a turnaround height measured **above field elevation** rather than above sea level, and slower than a turnaround speed. A break is then declared two ways, and the arm fires on either: at the sample that ends a reception gap, when the previous sample was stationary at an aerodrome and at least the dwell threshold has passed; or at the first sample that moves again after a continuous stationary run lasting at least the dwell threshold. Where the airport join finds nothing, “near an aerodrome” is false and the arm cannot fire, falling back to its floor. Because the arm's entire thesis is that legacy's low-altitude rule is the wrong test, that rule is deliberately *not* inherited — only the general “a long enough silence is a new track” rule is, which no arm disputes and which guarantees no airframe can end up with a month-long track.

What the measurement said. 86.15% clean on 2025 and 89.03% on 2024 — the closest any domain-knowledge arm gets to A4, and still short of it. Real geometry is worth a lot; it is worth less than not fragmenting on blanks.

A7 vertical_profile — a negative result, correctly attributed

What it changes. Groups on `icao24`, and adds a break based purely on the altitude trace, with legacy's gap rules as a floor.

What it is trying to fix. The case where the transponder never reports `on_ground` at all. If a flight is one climb–cruise–descent cycle, then a descent to circuit height followed by a climb back through it is a new sortie, whether or not there was a reception gap and whether or not ground contact was ever broadcast.

The rule. Both halves are required. First, *the descent*: some earlier sample of the group crossed the floor downwards — the previous sample above it, this one at or below. Second, *the climb back*: this sample is above the floor and the previous one is not. The break is placed at the upward crossing, so the departing leg begins there. The floor is measured **above field elevation**; comparing against sea level made the arm degenerate to bare legacy at any aerodrome above the floor’s own height, since a parked aircraft there already reads above it and no crossing is ever seen.

What the measurement said. 37.62% clean, last of the eight on both periods. An earlier implementation split on *any* upward crossing, which is not the idea, so its poor score could have been the idea failing or the code not being the idea. Implemented properly, on height above field, it scores *worse* still — which is what makes this a negative result rather than an unexplained number. The mechanism is visible in the boundary error: every percentile of its signed departure offset is positive (tenth percentile +19 s), meaning it systematically starts tracks late, having sliced off each departure’s initial climb.

Scored [gate to gate](#) it loses only -0.89 points on 2025 and -0.74 on 2024, among the smallest drops of any arm. An earlier draft of this study predicted the reverse — that an arm this poor airborne would be worse again once the turnaround counted — and the measurement refuses it. A7 groups on `icao24`, so no blank callsign at the stand can split its ground samples, and its own break fires at a vertical crossing, which happens in the air. It is already last of the eight before any taxi sample is scored, and the taxi adds almost nothing to that. A7 is a bad rule for a reason the gate interval cannot see, which is worth knowing about the gate interval as well as about A7.

A8 recommended — the two fixes, kept separate

This is the rule that now ships as `segmentation.method = "standard"`. **Its two changes are independent, they fix different failures, and the arm only works because they are kept separate**, so they are set out one at a time before any number.

What it changes. The group key loses its callsign column, and the break predicate gains a callsign test. Legacy’s two gap rules are kept underneath both, unchanged. Nothing about the gap behaviour of published tracks changes; what changes is which samples may share a track, and where a shared group is cut.

Fix one: take callsign out of the group key

The failure it addresses is A0's: a sample with no callsign forms its own group, so a hole in the data becomes a separate flight. That is where legacy's fragmentation comes from, and A4 showed that deleting the column removes it.

The change is exactly A4's: group on `icao24` alone. A blank or flickering callsign can no longer start a track, because callsign is no longer part of what makes a track a track.

What it costs on its own is A4's cost: two consecutive flights of one airframe now have nothing to separate them but a gap, and where the aircraft kept broadcasting through the turnaround there is no sufficient gap. They merge.

Fix two: break when the callsign genuinely changes

The failure it addresses is the one fix one creates. Legacy separated those two flights for the wrong reason — callsign was in the key, so any change of it started a new group — but it did separate them.

The change restores that separation as an explicit break rather than as a side effect of the key. Walking an airframe's samples in time order, carry forward the last **non-blank** callsign seen. Break at a sample when that sample's callsign is itself non-blank, the carried-forward one is non-blank, and the two differ. Comparing the raw callsign column instead would break at every blank sample and reproduce legacy's fragmentation exactly; it is carrying the last *real* value across the blanks that makes the test mean “the callsign changed” rather than “the callsign was missing”. Callsigns are compared trimmed, because ADS-B pads them to eight characters and `ABC123` is not a different flight from `ABC123`.

Why the two are genuinely independent: one is about which samples may share a track, the other about where a shared group is cut. The evidence that they are is that the combination beats **both** of the arms it draws on, rather than landing between them — which is what would happen if they were two attempts at the same thing.

The subtlety: the lookback must be bounded

The break predicate runs over the whole airframe's samples and has no notion of the track boundaries it is in the middle of deciding, so “the last non-blank callsign” is by default the last one *anywhere earlier in the month*. That reaches back across a gap break into the previous flight, and the next flight — whose first samples happen to be blank — is compared against a callsign belonging to a flight that already ended, and split in its own middle.

The fix bounds the lookback: the carried-forward callsign counts only if it is no older than the general gap threshold. That bound is the right one rather than an arbitrary one, because a silence longer than the gap threshold *is already a break*, so there is nothing legitimate to carry across it.

It is not a detail. Measured during implementation, the unbounded version fragmented 37.75% of flights, where `airframe_only` — which does no callsign splitting at all — fragments 6.90%, and where only 3.2% of tracks contain a genuine callsign transition to fire on. The mismatch between those two is what exposed the bug: a rule reaching one track in thirty cannot legitimately cost thirty points of fragmentation.

What the measurement said

90.80% clean on 2025 and 91.75% on 2024 — the best of the eight on both periods, and +40.65 and +38.96 points above `legacy`. Fragmentation falls from 49.44% to 8.45%.

The result that settles it is merging: 0.75%, against `airframe_only`'s 4.53%. Fix two recovers almost all of the merging fix one introduces, but not all: the remainder is still above `legacy`'s 0.41%. That residual is small, real, and where the downstream cost measured in [the payoff](#) comes from.

Tuning what actually ships

A **sweep** is a re-run of one rule at many parameter settings; a **cell** is one such setting. Earlier drafts of this study swept `legacy`, which answered a question about a rule nobody now runs. This sweep is of `recommended`, at the same four parameters production exposes:

- **gap_minutes** — a gap longer than this starts a new track, at any altitude.
- **low_alt_gap_minutes** — a shorter gap that also starts a new track, but only when the aircraft is low.
- **low_alt_ft** — the altitude below which the shorter gap applies.
- **callsign_lookback_minutes** — how far back the carried-forward non-blank callsign may reach before it stops counting. This axis exists only on `recommended`, and only since it was made settable; before that it was pinned to `gap_minutes` in the code.

The grid, and where each axis peaks

Stage 1 evaluated 235 cells across the first three axes on one day, with the lookback left following `gap_minutes`. Locating an optimum and quoting it are different claims, so the location is done on one day and the quoting on three.

End to end the grid spans 21.41 points of clean matching, from 69.58% to 90.98%. The parameters are not decorative: a rule handed the worst corner of its own grid gives up 53% of what the identity fix bought.

Table 3: For each value of each parameter, the best clean-match rate achieved anywhere in the grid at that value. A star marks each axis’s peak.

axis	value	best clean % at this value
gap_minutes (min)	10	84.601
gap_minutes (min)	15	86.705
gap_minutes (min)	20	88.614
gap_minutes (min)	25	90.064
gap_minutes (min)	30	90.830
gap_minutes (min)	40	90.985 *
gap_minutes (min)	50	88.661
gap_minutes (min)	60	85.427
gap_minutes (min)	90	73.291
low_alt_gap_minutes (min)	3	90.722
low_alt_gap_minutes (min)	5	90.751
low_alt_gap_minutes (min)	10	90.883
low_alt_gap_minutes (min)	15	90.957
low_alt_gap_minutes (min)	20	90.985 *
low_alt_gap_minutes (min)	30	90.953
low_alt_ft (ft)	1,000	90.791
low_alt_ft (ft)	2,500	90.884
low_alt_ft (ft)	5,000	90.969
low_alt_ft (ft)	7,500	90.982
low_alt_ft (ft)	10,000	90.985 *
low_alt_ft (ft)	15,000	90.958
low_alt_ft (ft)	20,000	90.881
low_alt_ft (ft)	30,000	90.711

Read down each block: every axis rises to a peak and falls again on the far side. That is what an **interior optimum** means, and it is the condition for calling a result an optimum at all. An optimum sitting on the edge of a grid is not an optimum — it is where you stopped looking.

The extension, and why it was necessary

`low_alt_ft` did not start out interior. The first grid ran it to 10,000 ft, and its best value was the largest tried with the curve still rising, which is the edge case above. This pipeline’s own history contains a sweep that stopped early and was read as “higher is always better”, so the grid was extended rather than the result reported.

Table 4: The low-altitude ceiling before and after the grid was extended. On the original grid the best value was the largest tried; extended, the curve turns over.

low_alt_ft	grid	best clean %
1,000	original	90.791
2,500	original	90.884
5,000	original	90.969
7,500	original	90.982
10,000	original	90.985
15,000	extension	90.958
20,000	extension	90.881
30,000	extension	90.711

Extended, the ceiling peaks at 10,000 ft and falls away to 30,000 — as it must, since past cruise altitude every sample counts as “low”, the two-threshold rule collapses into one, and `low_alt_gap_minutes` simply replaces `gap_minutes` everywhere.

The lookback axis

`callsign_lookback_minutes` is the axis unique to `recommended`, and until recently it was not an axis at all: the implementation reached back exactly `gap_minutes` because [that bound is the defensible one](#) — a silence longer than the gap threshold is already a break, so there is nothing legitimate to carry across it. The argument is sound and it was never measured. Making the parameter settable turns it into a question with an answer.

It was swept twice, at two different cells, because a lookback that is best at one gap threshold need not be best at another.

Table 5: The lookback axis at two cells of the three-parameter grid, on the full 2025 sample.

cell	callsign_lookback_minutes	clean %
gap 30 / low 15 / 5,000 ft (shipped)	15	88.432
gap 30 / low 15 / 5,000 ft (shipped)	20	88.561
gap 30 / low 15 / 5,000 ft (shipped)	25	89.146
gap 30 / low 15 / 5,000 ft (shipped)	30	90.803
gap 30 / low 15 / 5,000 ft (shipped)	35	90.892
gap 30 / low 15 / 5,000 ft (shipped)	40	90.610
gap 30 / low 15 / 5,000 ft (shipped)	50	88.812
gap 40 / low 20 / 10,000 ft	0	82.885
gap 40 / low 20 / 10,000 ft	5	82.795
gap 40 / low 20 / 10,000 ft	10	82.842

cell	callsign_lookback_minutes	clean %
gap 40 / low 20 / 10,000 ft	15	82.911
gap 40 / low 20 / 10,000 ft	20	83.029
gap 40 / low 20 / 10,000 ft	25	83.546
gap 40 / low 20 / 10,000 ft	30	85.054
gap 40 / low 20 / 10,000 ft	35	87.592
gap 40 / low 20 / 10,000 ft	40	90.985
gap 40 / low 20 / 10,000 ft	45	90.707
gap 40 / low 20 / 10,000 ft	50	89.678
gap 40 / low 20 / 10,000 ft	60	87.006
gap 40 / low 20 / 10,000 ft	120	73.399

Both curves peak at, or one step above, their own `gap_minutes`: 40 minutes at the tuned cell's gap of 40, and 35 minutes at the shipped cell's gap of 30. Decoupling the two buys 0.089 points at the shipped cell and 0.000 at the tuned one. So the coupling that was in the code by accident was very nearly the right choice, and `callsign_lookback_minutes` ships as `None`, meaning *follow `gap_minutes`*. The argument for the bound now has a number beside it.

What the sweep is worth against what shipped

Stage 2 and the 2024 confirmation re-measured the interesting cells on the full three-day samples, so a winner is chosen out of the day it was tuned on.

The shipped setting — gap 30, low-altitude gap 15, ceiling 5,000 ft — ranks 22 of 235 on the 2025 grid, and the best cell anywhere in it beats the shipped one by **0.18** points. **The sweep does not beat the defaults by enough to move them.** That is the result, and a sweep confirming a setting is as much a measurement as a sweep overturning one.

Then 2024, which is why nothing was re-shipped.

Table 6: 2024, the eight cells spanning the choice 2025 wanted to change. Every cell at gap 30 beats every cell at gap 40 — the reverse of 2025's ordering.

gap_minutes	low_alt_gap_minutes	low_alt_ft	clean %	
30	15	10,000	91.770	
30	15	5,000	91.750	shipped
30	20	10,000	91.740	
30	20	5,000	91.725	
40	15	10,000	91.231	
40	20	10,000	91.200	
40	15	5,000	91.185	

gap_minutes	low_alt_gap_minutes	low_alt_ft	clean %
40	20	5,000	91.159

On 2024 the `gap_minutes` ordering **reverses**: every cell at 30 minutes beats every cell at 40, and the shipped cell scores 91.75% against 91.77% for the best of the eight — 0.02 points, on the same `low_alt_gap_minutes` and a different ceiling. Retuning to 2025’s preferred gap would have cost 2024 about 0.54 points.

That is the closing argument. The gain 2025 offers is 0.18 points and does not survive the second sample, where the same change costs 0.54. The shipped setting is within a fifth of a point on the period that prefers something else, and preferred outright on the period that does not.

What a better segmentation buys ADEP/ADES

Each arm’s assignment is fed to the V7 flight list with **only `track_id` varying** — same input table, same detection configuration, same reference data — so any difference in the result is attributable to the segmentation.

Three terms, because the flight list scores differently from the ladder:

- **coverage** — the share of ground-truth flights the flight list gives *any* aerodrome for, in that role;
- **accuracy** — of those it does answer, the share it gets right;
- **score** — $\text{correct} - k \times \text{wrong}$ at $k = 2$, so a wrong answer costs twice what a right one earns and a silence costs nothing. **ADEP + ADES** is the sum over both roles, and it is the single number the tables below rank on.

The ground truth here is the flight-list studies’ own, keyed on callsign and day, which is why its denominator (95,116 flights on 2025) differs from the ladder’s. The two are not interchangeable.

As a control, `legacy` run through this harness scores 134,176 against the V7 flight-list study’s own published 134,168 for the same sample — a difference of 8, or 0.006%. That residual is itself informative and is explained in [Limitations](#).

The flight-list configuration these numbers were produced at

Every arm is scored at one fixed ADEP/ADES configuration. Naming it matters, because “the V7 recommended configuration” is a moving target and these numbers belong to one point on it. The table is read from the run’s own recorded parameters rather than typed, so it cannot drift from what was executed.

Table 7: The ADEP/ADES configuration every arm in this section was scored at, identical across all eight arms and both periods.

parameter	value	what it does
<code>adep_mode</code>	<code>endpoint</code>	Which algorithm names the departure aerodrome.
<code>ades_mode</code>	<code>trend</code>	Which algorithm names the destination aerodrome.
<code>endpoint_radius_nm</code>	30	endpoint : how close the track’s first or last fix must be to an aerodrome.
<code>endpoint_height_ft</code>	5000	endpoint : how high that fix may be, above field elevation.
<code>endpoint_sched_penalty_nm</code>	10	Distance added to an aerodrome without scheduled service, so a military field must be clearly nearest.
<code>endpoint_candidate_radius_nm</code>	100	How far the cached candidate table reaches. Wider than any detection radius, so the radius stays sweepable.
<code>trend_max_fl</code>	60	trend : only samples below this flight level vote.
<code>trend_radius_nm</code>	30	trend : how close a sample must be to an aerodrome to vote for it.
<code>trend_radius_exact</code>	<code>True</code>	Cut that radius on exact great-circle distance rather than on H3 band.
<code>trend_rank_by</code>	<code>haversine</code>	How trend chooses among candidates: exact distance, not H3 ring count.
<code>trend_vote_margin</code>	0	How far one direction must beat the other before the vote counts. Zero is a simple majority.
<code>trend_sched_penalty_nm</code>	10	The scheduled-service penalty, on trend ’s side.
<code>trend_smooth_half_window</code>	2	Half-width in samples of the rolling mean over barometric altitude.
<code>trend_smooth_before_cut</code>	<code>True</code>	Smooth altitude before applying the flight-level cut, not after.
<code>trend_bearing_tiebreak_nm</code>	20	Within this distance band, candidates are separated by course alignment rather than distance.
<code>trend_oaa</code>	<code>False</code>	Whether trend may emit the out-of-area marker. Off: on arrivals it is right barely half the time.
<code>k</code>	2	The scoring exchange rate: one wrong answer costs <code>k</code> right ones.

Cross-checked, not copied. Every value above was compared field by field against `DetectionConfig()` in the checkout these runs were made from, and they agree. Compared against the *v6.2* study’s shipped configuration, one difference is real and is reported rather than smoothed over:

v6.2 gives **trend**’s altitude cut a **datum**, defaulting to field elevation: the cut becomes `trend_max_height_ft` above the field rather than `trend_max_fl` above mean sea level, and `trend_max_fl` is retained but inert. That change landed on **main** **after** this study’s base commit and is not present in the checkout these payoff

numbers were produced from. The runs above therefore cut on FL60 above sea level.

This does not affect any comparison on this page: every arm is scored at the same configuration, so the datum is held constant across all of them. It does mean these absolute ADEP/ADES totals are not comparable with figures published by v6.2 or later, and it is the reason the payoff figures on this page are **not regenerated at this revision**. The V7 ladder that produced them cannot currently be re-run — its last rung no longer matches the shipped `DetectionConfig()` — so the choice is between quoting a measurement made at a configuration that has since moved and quoting one made at a configuration that answers a different question. The first is quoted, and labelled. [Limitations](#) carries it as a standing caveat rather than a footnote here.

Everything else on this page did move at this revision, and not because of any segmentation change. Ground truth improved underneath it: the APDF join now keys on the Samad flight identifier and the arrival-time correction is applied, which took the APDF-measured subset from about 22,000 flights to 35,398 on 2025. Every rate computed against it shifted slightly — the **recommended** — **legacy** gap by under 0.6 points, changing no ordering and no conclusion. The payoff figures alone are frozen at the older ground truth as well as at the older datum, which is what “retained historical” means where it appears below.

Segmentation quality and downstream score do not line up

This subsection is the one place all eight arms still appear. It reads the eight-arm CSVs already in `data/`, recomputes nothing, and is retained because it is the evidence for not optimising `clean_match_pct` directly — a historical measurement kept for what it rules out, not re-run.

Table 8: 2025. Ranked on ADEP+ADES score. The second column is the same arm’s clean-match rate from the ladder, so the two rankings can be read against each other in one place.

arm	clean % (ladder)	ADEP+ADES legacy	vs legacy	ADEP cov %	ADEP acc %	ADES cov %	ADES acc %
legacy	50.15	134,176	0	79.97	98.21	68.30	98.58
no_month_suffix	50.15	134,161	-15	79.97	98.21	68.30	98.57
recommended	90.80	133,445	-731	80.09	98.24	67.60	98.45
airframe_only	88.56	126,310	-7,866	78.29	97.00	66.97	97.30
traffic_style	46.37	125,972	-8,204	77.63	96.94	68.85	96.65
airport_anchored	86.15	124,105	-10,071	78.52	97.15	64.47	97.00
ground_anchored	80.11	117,090	-17,086	79.61	98.40	51.10	97.52
vertical_profile	37.62	100,472	-33,704	80.25	98.38	31.00	98.14

The two rankings disagree. `recommended` is first on the ladder and third here; `vertical_profile` is last on the ladder and has the **highest ADEP score of any arm**. A reader could take that to mean better segmentation makes the product worse. It does not, and the table says why in its last four columns.

Accuracy barely moves. Every arm lands within a couple of points on both roles. No arm is naming *wrong* aerodromes at a materially different rate.

Coverage moves, and almost all of the movement is on the arrival side. ADEP coverage spans a narrow band across all eight arms; ADES coverage runs from 31.0% to 68.9%. So the whole ranking is decided by how often the arrival algorithm can answer at all.

Why the arrival side is the sensitive one is not settled here. What the data says is consistent across both periods: the two arms with the worst arrival coverage by a wide margin, `ground_anchored` and `vertical_profile`, are the two whose breaks are placed at *low-altitude events*, while `legacy` and `recommended` place theirs at reception gaps and land close together. The destination algorithm reads its evidence from the low-altitude part of a track, so an arm that cuts there is cutting into the evidence. Isolating the mechanism would need a per-flight diagnosis this study did not run, so it is recorded as a hypothesis.

The lesson does not depend on that hypothesis: **an arm that is good at recovering flights can still be bad at preserving the evidence a downstream detector needs**, and the two have to be measured separately.

The labelling bug this study found, and why it is now history

The measurement above could not be made until a defect in the flight list was fixed. It is recorded because anyone meeting an OPDI flight list built before the fix needs to recognise it.

What it was. The flight list built one flight per `track_id` and labelled it with `F.min("flight_id")` — the lexicographically smallest callsign in the track. A blank sorts first.

Why it was harmless, until it was not. While callsign is part of the track's group key, every track carries exactly one callsign by construction, so that aggregate can never lose anything. It is correct only because of a property `legacy` supplies *by accident*: blank-callsign samples sit in tracks of their own. Any fix to `legacy`'s fragmentation necessarily destroys that property — which is the whole point of the fix. Drop callsign from the key and a track that carries a real callsign alongside blank ones is labelled blank; the flight becomes unidentifiable and matches no ground-truth counterpart.

What it cost. Measured against the pipeline as it then stood, adopting the better segmentation *lost* ADEP/ADES points, and the losing arms lost them by coverage collapse with accuracy unmoved — the signature of a labelling failure rather than a geometric one. `Legacy`'s

fragmentation and the downstream coverage collapse were one phenomenon seen from two ends, so the recommendation was conditional: fix the label first, then ship the segmentation.

What was actually wrong was larger than the label. Tracing it found that `flight_id` is used as a grouping or join key in a dozen places downstream and is never aggregated to one value per track — so the flight list is written on the invariant *one `flight_id` per `track_id`*, which `legacy` guarantees and the new segmentation breaks. A track carrying several callsigns does not merely get labelled wrongly; it *fans out into several rows* at each of those keys. The same latent fan-out was present in the event extraction of step 04.

It is fixed. Every point where the flight list and the event extraction read the track table now resolves `flight_id` to the track’s dominant non-blank callsign, over the unfiltered frame, before anything groups or joins on it. Runs that stamp a legacy version string skip the resolution, so re-running a released month still reproduces what was released, column for column. The numbers throughout this page are from the fixed pipeline; there is no repaired-versus- unrepaired pairing to interpret any more.

The convergence check

The benchmark kept its own independent repair, relabelling each track with its most frequent non-blank callsign inside the harness, and both jobs are still run. Now that production does the same thing, their **agreement** verifies that the production fix and the benchmark repair are the same operation. A disagreement would be a finding rather than a measurement.

Table 9: The two jobs agree where it matters. `recommended` comes back identical on every recorded field in both periods; before the production fix the two differed by tens of thousands of flights.

period	arm	production fix + harness		
		production fix only	repair	difference
2025	legacy (control)	134,176	134,164	-12
2025	recommended	133,445	133,445	0
2024	legacy (control)	128,932	128,932	0
2024	recommended	128,115	128,115	0

`recommended` agrees exactly, on both periods, which is the check passing.

`legacy` does not, on 2025 only, and by 12 points confined entirely to the arrival side — the same flights answered, a handful labelled with a different aerodrome. That was traced, and it is not the callsign repair: it is a non-deterministic aggregate in shipped ADES detection, which is reported in [Limitations](#) because it is a real defect and it is not this study’s to fix.

Where the residual comes from

With the labelling fixed in production, the recommended segmentation scores **133,445** against legacy's **134,164** on 2025 and **128,115** against **128,932** on 2024 — short by 719 and 817 points, 0.54% and 0.63% — while recovering 41 points more flights on the ladder. Its **ADEP** coverage slightly *exceeds* legacy's on both periods: 80.09% against 79.97%.

The residual is not accuracy and it is not departures. It is arrival coverage, and the only failure mode that grew is the one [A8](#) said would grow: **recommended** merges 0.75% of flights where **legacy** merges 0.41%, and a merged track has one arrival where there should be two, so one of the two flights loses its destination outright. The two statistics are measured against different ground truths — the ladder's and the flight list's — so they cannot be reconciled flight for flight, and merging on its own is smaller than the coverage difference it is offered to explain. What can be said without qualification is that merging is the only thing that moved in the wrong direction, and that it moved.

That is the price of grouping on the airframe. It is stated rather than rounded away, and it is small enough that the 41-point gain in recovered flights buys it many times over.

What shipped

The A8 rule ships, as `segmentation.method = "standard"`: group on `icao24`, break when the last non-blank callsign genuinely changes with the lookback bounded to the gap threshold, and keep legacy's gap rules as a floor.

At these four values, beside what [the sweep](#) preferred on each period:

Table 10: The shipped values against the cell each period's grid preferred. The lookback column is read at the shipped cell of the other three axes.

parameter	shipped	best on 2025	best on 2024
<code>gap_minutes</code>	30	40	30
<code>low_alt_gap_minutes</code>	15	20	15
<code>low_alt_ft</code>	5,000	10,000	10,000
<code>callsign_lookback_minutes</code>	None — follows <code>gap_minutes</code>	35	not swept

What the differences cost, measured. Moving all three gap parameters to 2025's preferred cell is worth 0.18 points on 2025; decoupling the lookback from `gap_minutes` and setting it to 35 minutes is worth a further 0.089. On 2024 the shipped values of `gap_minutes` and `low_alt_gap_minutes` are the preferred ones outright, and the ceiling costs 0.02 points.

Nothing was re-shipped, for one reason: 2025’s preferred `gap_minutes` is beaten by the shipped value on 2024 by 0.54 points, so adopting the 2025 optimum would trade a fifth of a point on one sample for half a point on the other.

The recommendation was conditional when this study first reported, and the condition was an ordering rather than a hedge. Both steps have now been taken, in this order:

1. **First, the flight-list labelling was fixed.** `flights.py` no longer picks a track’s callsign with `F.min`; every read of the track table resolves `flight_id` to the track’s dominant non-blank callsign before anything groups or joins on it, and the same fix was applied to step 04’s event extraction. This was worth doing on its own merits even had the segmentation never changed, because it is a silent trap for any future change to track identity and it fails as a *coverage* statistic rather than as an error — nothing looks broken, there is simply less data.
2. **Then the segmentation was switched.** Reversed, the pipeline would have spent the interval between the two changes at substantially reduced ADEP coverage.

Three things a consumer needs to know, all of them consequences of the release rather than open questions:

- **track_id changes for all data produced from this release forward**, in shape as well as value: `standard` carries no `_{year}_{month}` suffix, so identifiers become `{hash}_{offset}`. A consumer joining on `track_id` across the boundary gets an empty join rather than an error, which reads as missing data; a consumer parsing the suffix breaks outright.
- **Past months do not reproduce under the new rule, and are not meant to.** Setting `segmentation.method` back to "legacy" reproduces the published identifiers, and the version-guarded paths in the flight list and the event extraction reproduce the published labels alongside them.
- **Nothing in the published data records which rule produced a row.** `osn_tracks` carries no version column, so the only way to tell is to know when the row was published. This is a deliberate decision and its cost is written down in [Limitations](#).

Limitations

*The claims this page does **not** make, and the defects it found and did not fix.*

The baseline is a re-implementation, not the published `track_id`

Two distinct things are meant by “the baseline”, and neither is what production now runs.

First, this study measures algorithms in a harness rather than the pipeline itself. A0 is a re-implementation of the legacy *rule* inside the generic segmentation engine, which is asserted by

test to reproduce the frozen algorithm’s partition under production parameters. That contract is on the partition, not a claim to have run the pipeline.

Second, applied to *cleaned* tracks, the legacy rule does not reproduce the published identifiers exactly: it differs on 909 of 236,899 tracks, 0.38%, and on 8 of this page’s 134,176 ADEP/ADES points — the control residual noted in [the payoff](#). Production’s identifiers were computed upstream on raw altitudes, while `osn_tracks_clean` masks bad altitudes to NULL — and `NULL < 5000` is NULL, not true, so the low-altitude split silently does not fire where it once did. Measured, **950 of 950 disagreements have a NULL barometric altitude**, against a 21.4% base rate. The published `osn_tracks_clean` therefore carries a `track_id` that its own columns can no longer reproduce: nothing is wrong with the published data, but anyone recomputing identifiers from the clean table — the obvious thing to do — gets different tracks and no warning.

Neither point affects the comparisons on this page, because every arm reads the same cleaned table through the same engine and is subject to both behaviours equally. Version 2 of this study is the one that runs the pipeline end to end.

Published data carries no marker of which segmentation produced it

`osn_tracks` has no version column, so **no row of published OPDI data says which segmentation rule built its `track_id`**. Distinguishing the two regimes depends entirely on knowing the release date.

The decision not to add such a column was deliberate and its cost is concrete. The guards that let a legacy re-run reproduce released labels — in `flights.py` and in `events.py` — key on the *run’s configuration*, not on the data: they ask “is this run stamping a legacy version string?” when the question they need answered is “were the tracks I am reading built with the legacy rule?”. Nothing in the data can tell them. So a legacy-configured run over tracks that were rebuilt with `standard` will take the legacy branch and publish under a frozen version string, and no check anywhere can catch it. The same blind spot reaches the flight-events ladder, whose lower rungs carry a legacy events version and would silently measure the new segmentation if re-run after the switch.

ADES values are not reproducible run to run

This is a defect in shipped ADEP/ADES detection, it predates this study, and it is not fixed. It is recorded here because both this page and the flight-list studies quote ADES percentages to two decimals, and a figure that is not reproducible must say so.

`flights.py:_rank_airports` ranks candidate aerodromes correctly with `row_number()`, and then names the winner with `first(apt_ident)` — an aggregate with no defined order. Adding a join upstream can change the answer. The defect is self-detecting, because the *runner-up*

list is built from the rank rather than from `first()`, so a wrongly named aerodrome shows up in its own “not most likely” list.

Measured across repeated runs on identical code and data: roughly 28 of 88,737 named ADES per run, about 0.03%, come from the undefined-order aggregate, and about 20 can flip between two runs. The affected flights are short hops between neighbouring aerodromes. **ADEP is immune** — the departure path filters on the rank instead of aggregating, which is the correct pattern and already exists in the same file. `ades_mode` defaults to `trend`, so this is the shipped arrival path.

It is the explanation for the one disagreement in [the convergence check](#): the control differs by a handful of ADES flights on one period and by none on the other, with coverage identical to six decimals.

Two ground-truth intervals over twenty hours survive in the 2024 sample

Ground truth is built from Network Manager’s own AOBT_3/ARVT_3 pairing, and two intervals in the 2024 sample exceed twenty hours — the longest being 27.9 h, THY801 LTFM→SKBO. Because samples are attributed to a flight by containment, an over-long interval can absorb samples belonging to a neighbouring leg of the same airframe.

These are **not** the defect that was fixed. A separate loader defect, since corrected, windowed ground truth on the off-block day rather than on the flight interval and keyed the APDF departure join on the off-block day rather than the take-off day; the second let a nightly service take its take-off time from yesterday’s rotation, producing intervals of up to 26.9 hours. Fixing it removed every such interval from 2025 and all but these two from 2024, and moved the scored flight counts by about a tenth of a percent with every rate moving by no more than 0.038 pp.

The two survivors are NM’s own pairing and are undiagnosed. They were deliberately not filtered out: a length filter would change the denominator of every metric on this page, which is a larger and less examined intervention than the contamination it removes. The effect is carried, stated, and small.

The ADEP/ADES configuration predates the v6.2 datum change

As [§ the flight-list configuration](#) records, the payoff runs were made from a checkout that does not contain v6.2’s altitude-cut datum, so `trend` cuts at FL60 above mean sea level rather than at a height above field elevation. The configuration is identical across all eight arms, so every comparison in [the payoff](#) is unaffected; the absolute totals are not comparable with figures published by v6.2 or later.

They are also not regenerated, and will not be until the V7 ladder can run again — its last rung no longer matches the shipped `DetectionConfig()`. So the payoff chapter alone quotes

a measurement made against the older ground truth and the older datum, while every other number on this page comes from the current one. That is a real inconsistency inside one paper. It is carried rather than hidden because the alternative is a re-run at a configuration that answers a different question, and because the quantity the chapter reports — the difference between two arms at one fixed configuration — is the quantity least disturbed by holding that configuration still.

Ground truth covers Network Manager flights only

Merges with traffic absent from the NM flight table — military, general aviation, non-ECAC operators — are invisible to the metric, because the merged partner has no ground-truth flight to be merged with. **The merge rates on this page are therefore floors, not estimates.** Fragmentation is unaffected, since it is measured within flights that ground truth does contain.

One arm is untested, not null

`no_month_suffix` is identical to legacy on both samples because both sit inside a single month. It is reported as untested. Evaluating it needs a sample spanning a month boundary.

Two three-day samples in June

Both samples are in early June. Nothing here licenses a seasonal claim, and the callsign statistics in particular could plausibly differ in a period with more charter or seasonal traffic.

The sweep covers one arm's parameters, not every arm's

[The sweep](#) varies the four parameters of **recommended**. It does not vary the ground-dwell or turnaround parameters that the domain arms introduce, so those arms are each compared at one setting and could be better at another. Nor does it sweep **legacy**: an earlier version of this study did, and found it within a fraction of a point of its own best cell, but those runs are not in this revision and the claim is not made here. What the ladder measures is each rule at one setting; what the sweep measures is how much one rule's setting is worth.

The payoff holds the flight list fixed

Each arm is scored at one flight-list configuration — the one tabulated in § [the flight-list configuration](#) — rather than re-tuned per arm. A joint optimum might be better for some arm; finding it would multiply cluster time by the number of arms, and one Spark job runs at a time. The natural follow-up, now that the labelling is fixed, is to re-tune the flight list on the recommended arm alone: one extra grid, not eight.